

Speculative Decoding for Verilog: Speed and Quality, All in One

Changran Xu^{1,3,†}, Yi Liu^{1,3,†}, Yunhao Zhou^{1,3}, Shan Huang^{2,3}, Ningyi Xu², and Qiang Xu^{1,3}

¹The Chinese University of Hong Kong, Shatin, Hong Kong S.A.R.

²Shanghai Jiao Tong University, Shanghai, China

³National Technology Innovation Center for EDA, Nanjing, Jiangsu, China

Abstract—The rapid advancement of large language models (LLMs) has revolutionized code generation tasks across various programming languages. However, the unique characteristics of programming languages, particularly those like Verilog with specific syntax and lower representation in training datasets, pose significant challenges for conventional tokenization and decoding approaches. In this paper, we introduce a novel application of speculative decoding for Verilog code generation, showing that it can improve both inference speed and output quality, effectively achieving speed and quality all in one.

Unlike standard LLM tokenization schemes, which often fragment meaningful code structures, our approach aligns decoding stops with syntactically significant tokens, making it easier for models to learn the token distribution. This refinement addresses inherent tokenization issues and enhances the model's ability to capture Verilog's logical constructs more effectively. Our experimental results show that our method achieves up to a 5.05× speedup in Verilog code generation and increases pass@10 functional accuracy on RTLLM by up to 17.19% compared to conventional training strategies. These findings highlight speculative decoding as a promising approach to bridge the quality gap in code generation for specialized programming languages.

Index Terms—Verilog code generation, speculative decoding

I. INTRODUCTION

The rapid advancement in large language models (LLMs) has transformed code generation across various programming languages [1]–[4]. These models, driven by advanced pre-training techniques, have achieved notable success in generating syntactically and semantically correct code for widely used languages like Python and C++. However, their application to specialized languages such as Verilog, which is critical for hardware design and verification, remains limited due to the unique challenges posed by Verilog's syntax intricacies and its underrepresentation in training datasets.

Unlike natural languages, programming languages like Verilog are governed by strict syntactic rules, where even minor deviations can lead to significant compilation failures or functional errors. Traditional tokenization methods, such as Byte Pair Encoding (BPE) [5], fragment meaningful code structures into subword units, obscuring the logical relationships inherent in Verilog. Consequently, models often struggle to accurately capture the syntax and semantics of Verilog code. Grammar-based approaches attempt to mitigate this issue by representing code as sequences of grammar rules [6]–[11]. While effective for smaller models and datasets, these methods face scalability issues when applied to LLMs, due to vocabulary explosion and distribution shifts. Moreover, these methods remain largely unexplored for underrepresented languages like Verilog, which presents an additional layer of complexity.

To address these challenges, we propose a novel application of speculative decoding [12]–[15] for Verilog code generation. While

[†] These authors contributed equally.

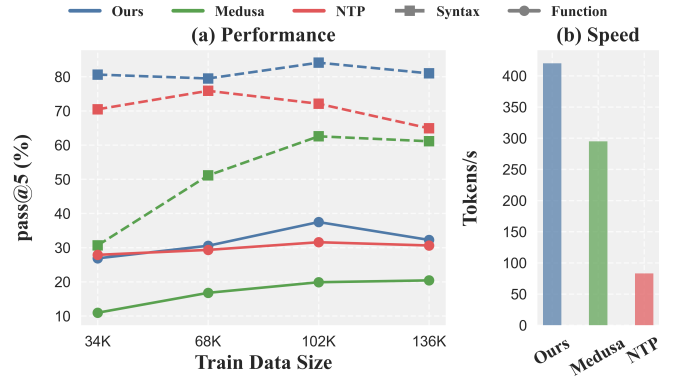


Fig. 1. A brief comparison of the performance and speed of our method against the MEDUSA method and the conventional next token prediction (NTP) approach. The experiments are conducted using the CodeLlama-7b model, with performance metrics evaluated on the RTLLM benchmark.

speculative decoding has traditionally been employed to accelerate LLM inference by predicting multiple tokens simultaneously to reduce latency, our key insight is that *speculative decoding, when carefully aligned with syntactically significant tokens, can also improve the quality of code generation*. By structuring decoding stops around meaningful fragments in Verilog, such as identifiers, keywords, and operators, we enable the model to better capture the logical and structural relationships inherent in the language. This alignment simplifies the learning of Verilog's token distribution, mitigating issues caused by conventional tokenization schemes and enhancing the syntactic and functional correctness of the generated code. Specifically, the main contributions of our work include:

- To the best of our knowledge, this is the first application of speculative decoding aimed not only at improving inference speed but also enhancing output quality.
- We propose a simple yet effective method for identifying syntactically significant tokens in Verilog using abstract syntax trees (ASTs) and modify the speculative decoding scheme by incorporating syntax-enriched labels to align decoding stops with these tokens.
- Our decoding scheme further unlocks the potential of the original MEDUSA method [15] by training it with dynamic labels in a flexible manner, increasing the number of effective heads and further accelerating the speedup.

We conduct experiments with two models, CodeT5p-220m-bimodal [2] and CodeLlama-7b [3]. The results show that our method achieves up to a 5.05× speedup in Verilog code generation and increases pass@10 functional accuracy on RTLLM [16] by up to 17.19% compared to conventional training strategies. Impressively,

II. RELATED WORK

A. LLM for Verilog Generation

A. LLM for Verilog Generation

B. Syntax-Aware Tokenization for Code

without significantly altering the distribution of code tokens. By introducing a novel application of speculative decoding for Verilog, our method can enhance both the speed and quality of Verilog code generation.

C. Speculative Decoding

As the size of language models continues to grow, inference latency has become a critical challenge for practical applications. To alleviate this issue, speculative decoding [12]–[14] has been proposed to reduce the number of decoding steps. Traditional speculative decoding employs a smaller draft model to generate an initial token sequence, which the original, larger model then refines to produce an acceptable continuation. However, it remains challenging to acquire and maintain a separate draft model. To address this, MEDUSA [15] introduces additional decoding heads that predict multiple tokens concurrently, enabling seamless integration into existing LLM systems. MEDUSA offers two fine-tuning approaches tailored to different use cases: (1) MEDUSA-1, which is fine-tuned on a frozen LLM, ensuring lossless acceleration; and (2) MEDUSA-2, which is jointly fine-tuned with the backbone LLM, achieving higher prediction accuracy and greater speedups. While originally developed for accelerating LLM inference, we find that speculative decoding can also preserve the syntactic structure of code by modifying its decoding mechanisms. By aligning decoding stops with syntactically significant tokens, speculative decoding enhances both inference speed and output quality in Verilog code generation. Fig. 1 presents a brief comparison of the performance and speed of our method against the original MEDUSA method and the conventional next token prediction (NTP) approach.

III. METHODOLOGY

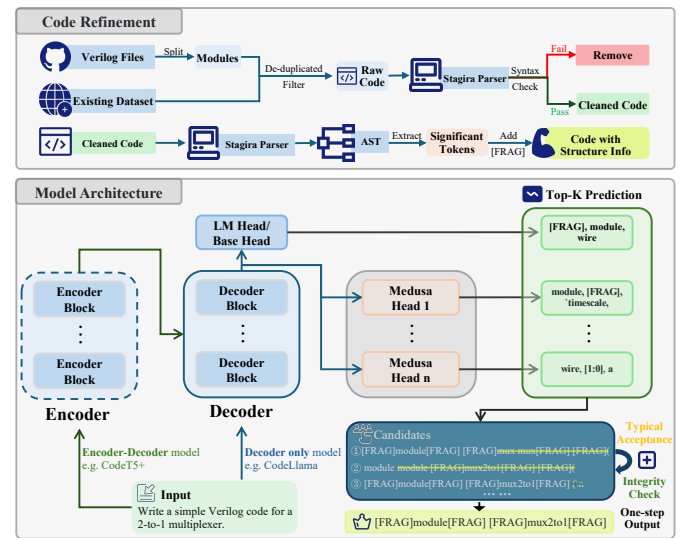


Fig. 2. The overview of the data refinement process and model architecture.

A. Dataset Construction

Our dataset comprises .v files collected from Github using Verilog as the search keyword. Each file is segmented into functional Verilog modules, and duplicates are removed using MinHash and Jaccard similarity metrics [31]. We also filter out files lacking complete module and endmodule structures or primarily consisting of comments. Additionally, we supplement our dataset with data from the open-source projects MG-Verilog [25] and RTLCoder [23], resulting a total of 13,6134 data items. To ensure the quality of the

dataset, we apply the Stagira Verilog parser [32] to perform syntax checks on all code samples and retain only those that pass (*i.e.*, cleaned code). For these cleaned code samples, we use the parser to generate their corresponding ASTs, from which the syntactically significant tokens are extracted. The entire code refinement process is illustrated in Fig. 2. For the cleaned code collected from Github, we leverage GPT-4 [33] to generate functional descriptions, while the data from MG-Verilog and RTLCode already include code summaries.

B. Model Architecture

In this work, we evaluate our approach using two base models: CodeT5p-220m-bimodal [2] and CodeLlama-7b [3], which differ in architecture and size. Specifically, CodeT5p employs an encoder-decoder architecture while CodeLlama is a decoder-only model. As shown in Fig. 2, we augment the base models with additional heads attached to their last hidden states to predict multiple tokens concurrently, following the original MEDUSA method [15]. To improve both inference speed and the output quality of Verilog code generation, we modify MEDUSA’s decoding mechanism to align decoding steps with syntactically significant tokens, which ensures that the model maintains a complete syntactic structure at each decoding step, enabling it to explicitly capture the Verilog syntax. For detailed explanations, please refer to Section III-C.

During the decoding process, given that the model is currently at position t , the base model will predict the token at $t + 1$, while the i -th head will predict the token at $t + i + 1$. Each token generated by the heads is first evaluated by the typical acceptance strategy [15]:

$$p_{\text{base}}(x_{t+i+1} | x_1, x_2, \dots, x_{t+i}) > \min(\epsilon, \delta \exp(-H(p_{\text{base}}(\cdot | x_1, x_2, \dots, x_{t+i})))) \quad (1)$$

where p_{base} represents the prediction probability of the base model, $H(\cdot)$ denotes the entropy function, and ϵ, δ are hyper-parameters. A token is accepted only if (1) holds for it and all preceding tokens. At this decoding step, suppose only the tokens predicted by the first u heads are accepted. These tokens, spanning positions $t + 1$ to $t + u + 1$, are then re-evaluated to ensure they form a complete syntactic structure. Any extraneous tokens that compromise the integrity of the code fragment are discarded. For instance, if the tokens from $t + 1$ to $t + v + 1$ (where $v < u$) already constitute a complete code fragment, such as an identifier or keyword, the outputs from the remaining heads, corresponding to $t + v + 2$ to $t + u + 1$, are discarded. During decoding, we maintain several candidates comprising the top- k predictions from the base model and additional heads. The final prediction for the current step is the *longest accepted prefix* among all candidates.

C. Syntax-Enriched Labels

In this section, we explain how syntactically significant tokens are identified and present a novel method to construct syntax-enriched labels for incorporating syntax into the speculative decoding process. To identify syntactically significant tokens, we first parse Verilog code into ASTs, from which we extract leaf nodes and non-terminal nodes that contains critical information as keywords. Additionally, we supplement these keywords with commonly used Verilog constructs, such as `posedge` and `endmodule`. Together, these keywords form the syntactically significant tokens. We then use regular expressions (regex) to segment the code into meaningful fragments that maintain syntax integrity based on these keywords. At each segmentation point, we insert a special token `[FRAG]` to prepare for constructing syntax-enriched labels. Fig. 3 illustrates this process with an example.

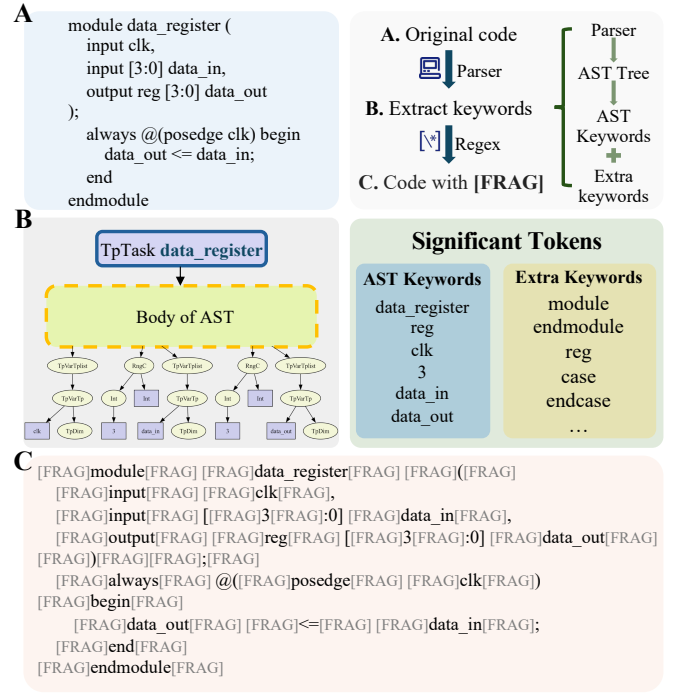


Fig. 3. An example demonstrating the identification and extraction of syntactically significant tokens from Verilog code.

To incorporate syntax information into the speculative decoding process, we modify the decoding mechanism by introducing syntax-enriched labels. Specifically, the label for the base model, denoted as L_0 with a length of `sequence_length`, is the tokenized version of the Verilog code filled with `[FRAG]` tokens. And the label for the i -th head is first derived by left-shifting the base model’s label by i , resulting in $L_i = L_0[i :]$. `[PAD]` tokens are then appended to ensure that the head labels align in length with the base model’s label. The labels for all heads are then concatenated with the base model’s label, forming the combined label *Labels* with size $(\text{num_heads} + 1) \times \text{sequence_length}$. As shown in Fig. 4, for each sequence position, we identify the position of the last `[FRAG]` token along the decoding heads. Labels beyond this position are replaced with `[IGNORE]` tokens, excluding them from loss computation. This ensures that, for any given sequence position s , the base model’s label and the corresponding head labels, *Labels*[:, s], represent syntactically meaningful fragments by excluding incomplete information. The syntax-enriched labels align decoding steps with syntactically significant tokens, enabling the model to better capture the syntactic structure of Verilog code. Additionally, the progressive increase of the proportion of `[IGNORE]` tokens in the labels of later heads reduces their prediction difficulty, improving the model’s training efficiency and allowing us to train more robust heads than the original MEDUSA method, which further accelerates inference. To optimize performance, we design a parallel algorithm that fully parallelizes the procedure, significantly speeding up the process and minimizing computational overhead during training. The pseudo-code for this algorithm is provided in Fig. 4.

IV. EXPERIMENTS

A. Experimental Setup

We select CodeLlama-7b-Instruct-hf [3] (CodeLlama) and CodeT5p-220m-bimodal [2] (CodeT5p) as our base models. For

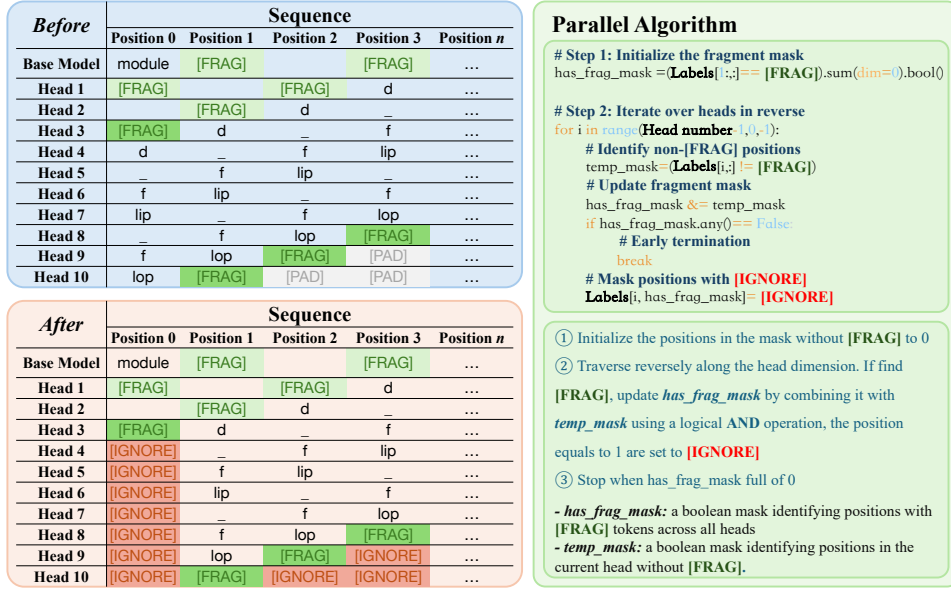


Fig. 4. The construction of syntax-enriched labels for aligning decoding steps with syntactically significant tokens. The top-left panel illustrates the initial labels of Verilog code filled with [FRAG] tokens, while the bottom-left panel depicts the final syntax-enriched labels used for training. The right panel presents the parallel algorithm for accelerating the label construction process.

each base model, we append 10 additional heads and fine-tune it for Verilog code generation, comparing our syntax-enriched training method (Ours) with the original MEDUSA-2 method [15] (Medusa) as one baseline. We also consider the conventional NTP scheme (NTP) as another baseline.

1) *Training Data*: The training data is constructed using the dataset introduced in Section III-A. Specifically, it is formatted into the Alpaca style [34], with the natural language description as input and the corresponding Verilog code as output, resulting in 136K samples for fine-tuning. Due to the 2048-token limit of the CodeT5p model, we exclude examples exceeding this threshold, resulting in 128K training samples for models based on the CodeT5p architecture. The data used for the two baseline methods is identical to ours, except that it does not incorporate the syntax-enriched labels. To evaluate model performance across varying training data sizes, we fine-tune the models not only on the full dataset but also on random subsets comprising 1/4, 1/2, and 3/4 of the original data.

2) *Model Training*: All model training is conducted on four NVIDIA A800-SXM4-80GB GPUs using the Distributed Data Parallel (DDP) module from PyTorch.

We utilize the Axolotl framework [35] to fine-tune models based on CodeLlama with QLoRA [36], using consistent hyperparameters: a LoRA adapter rank of 32, α set to 16, and a dropout rate of 0.05. Additional hyperparameters are also kept constant, including a micro-batch size of 1, an 8-bit AdamW optimizer with a cosine learning rate scheduler, an initial learning rate of $5e^{-4}$ for the base model, a warmup period of 40 steps, and a maximum sequence length of 8192 tokens. To improve training throughput, we employ the multipack method, which combines multiple short sequences into a single batch. For models based on CodeT5p, we fine-tune directly using a batch size of 2, the AdamW optimizer, a learning rate of $5e^{-4}$ for the base model, a warmup ratio of 0.1, and a maximum sequence length of 2048 tokens.

For models with additional heads, the learning rate for the decoding heads is set to four times that of the base model. The overall loss for

these multi-head models is computed using the method proposed in MEDUSA [15]:

$$\text{Loss} = \text{Loss}_{\text{base}} + \lambda \cdot \sum_{i=1}^n (\text{Loss}_{\text{head}_i} \cdot \gamma^i) \quad (2)$$

In this equation, λ is a dynamic weighting factor that adjusts the influence of the heads' losses on the overall loss. During training, λ follows a sine growth pattern, increasing from 0 to 0.2 as training progresses. The parameter γ , set to 0.8 in our experiments, serves as a decay coefficient to differentially weight each head's loss. Here, n represents the number of heads, fixed at 10 in our experiments.

3) *Model Inference*: During inference, all models are loaded in float16 format. CodeLlama-based models are configured with a maximum token length of 8192, while CodeT5p-based models are limited to 2048 tokens. For speed evaluation, each prompt is processed using two decoding methods: greedy decoding and sampling decoding at a temperature of 0.8. For quality evaluation of the generated Verilog code, 20 responses are sampled per prompt at temperatures of 0.2, 0.4, 0.6, and 0.8. The final result for each prompt is determined by selecting the output with the highest accuracy across all temperatures.

B. Evaluation Benchmark and Metric

We use RTLLM [16] and VGen [22] as evaluation benchmarks. Specifically, we employ low-level prompts from VGen that align with the format of our training data. These prompts describe the module's function along with its header, including the module name and the input and output types, which are the most challenging cases.

1) *Speed Evaluation*: In addition to prompts from RTLLM and VGen, we utilize GPT-4 to generate additional prompts for the Verilog code generation task based on the input prompt formats of RTLLM and VGen, aiming to enhance testing accuracy by increasing the diversity of prompts used during evaluation. Ultimately, the generation speed of the fine-tuned models is assessed using a total of 575 input prompts. For each prompt, the model generate outputs using both greedy decoding and sampling decoding methods, with

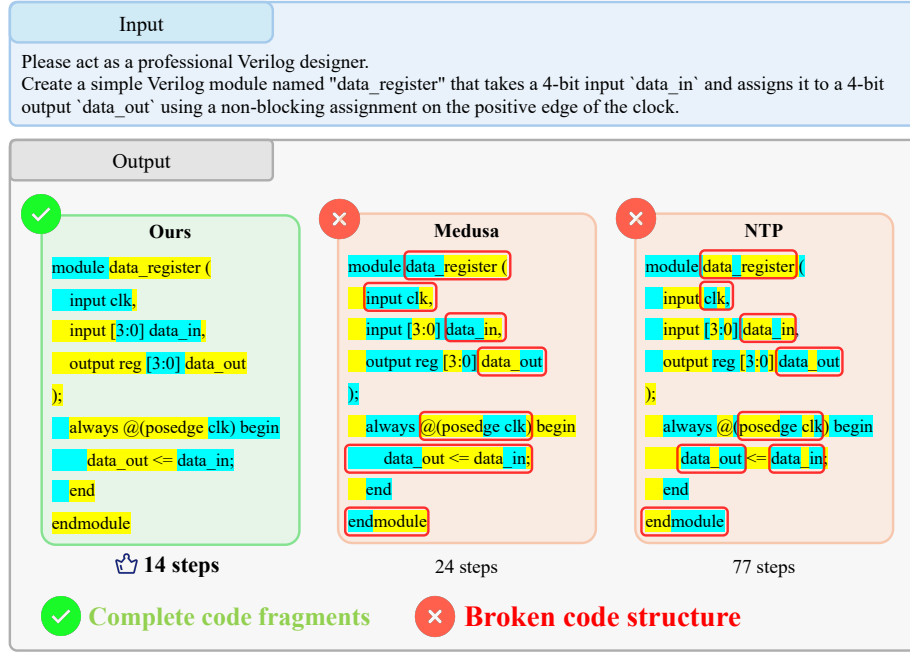


Fig. 5. The comparison of the decoding processes for a specific example using our method, Medusa, and NTP. Remarkably, our method generates the output in significantly fewer steps while preserving the integrity of syntactic structure at each decoding step.

the inference time recorded separately for each method. We then calculate the model's generation speed using the following formula:

$$Speed = \frac{1}{n} \sum_{i=1}^n \frac{Output\ Token\ Length_i}{Inference\ Time_i} \quad (3)$$

where n represents the total number of outputs generated (i.e., 575 outputs for each decoding method, resulting in 575×2 for the two decoding strategies).

The speedup of each fine-tuned model is calculated relative to its counterpart fine-tuned with NTP, which serves as the baseline, and is defined as follows:

$$Speedup = \frac{Speed\ of\ Fine-tuned\ Model}{Speed\ of\ Fine-tuned\ Model\ with\ NTP} \quad (4)$$

2) *Quality Evaluation*: For syntax evaluation, a design is considered syntactically correct if both the design and its testbench successfully compile together using iverilog [37]. For functionality evaluation, a design is deemed functionally correct if its output matches the expected results for all testbench-provided stimuli. We use the $pass@k$ metric, introduced in VerilogEval [20], to evaluate both the functional and syntactic correctness of Verilog code generated by LLMs. For a specific prompt i , $pass@k$ reflects the likelihood of at least one correct solution among k randomly selected attempts:

$$pass@k = \mathbb{E}_{prompt_i} \left[\frac{1 - \binom{n-c}{k}}{\binom{n}{k}} \right] \quad (5)$$

Here, n denotes the total number of samples generated by the model for each prompt, and c represents the count of outputs that pass the functional check. To ensure a comprehensive assessment while maintaining experimental efficiency, we set $n = 20$ for all prompts and evaluate k at values 1, 5, and 10.

We also use an additional evaluation criterion, *Pass Rate*, to further assess model performance. For each example in the benchmark, the model is considered successful if any of the 20 generated attempts

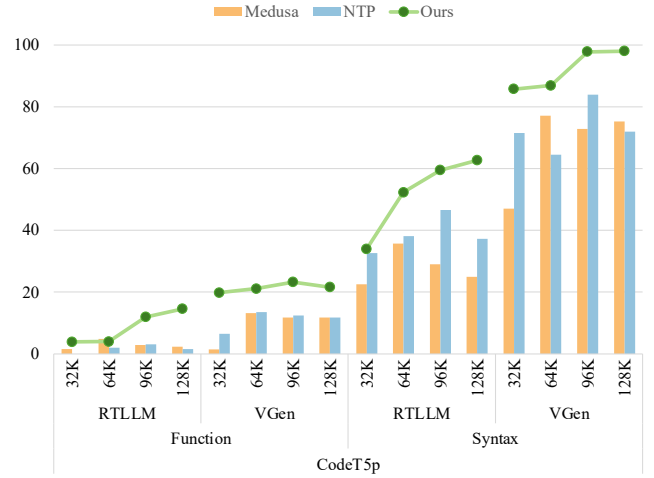


Fig. 6. The comparison of pass@5 output quality between our method and baselines using the CodeT5p architecture.

passes validation. Given m successful cases, the overall *Pass Rate* is calculated as:

$$Pass\ Rate = \frac{m}{len(Benchmark)} \quad (6)$$

C. Experimental Result

Table II highlights the superior inference speed achieved by models fine-tuned with our method compared to those trained using Medusa and NTP. The most significant improvement is observed with the CodeLlama model, which achieves a speed of 420.13 tokens per second, corresponding to a 5.05× speedup over the NTP baseline. For CodeT5p, our method delivers a 2.66× speedup, outperforming the model trained with Medusa, which achieves a 1.16× speedup. To

TABLE I
EVALUATION RESULTS FOR THE QUALITY OF GENERATED VERILOG CODE

Test	Model	Data Size	Benchmark	pass@1 (%)			pass@5 (%)			pass@10 (%)			Pass Rate (%)		
				Ours	Medusa	NTP	Ours	Medusa	NTP	Ours	Medusa	NTP	Ours	Medusa	NTP
Function	CodeLlama	34K	RTLLM	16.21	4.66	16.72	26.87	10.96	27.89	33.85	13.07	32.62	41.38	13.79	37.93
			VGen	30.59	27.06	29.12	49.17	34.48	47.65	56.78	38.04	55.16	64.71	41.18	58.82
		68K	RTLLM	18.28	13.28	18.79	30.55	16.78	29.34	35.52	19.87	34.24	41.38	24.14	37.93
			VGen	32.06	25.59	24.12	47.01	32.88	44.12	53.61	36.59	51.81	58.82	41.18	58.82
		102K	RTLLM	20.52	13.10	17.07	37.48	19.88	31.58	46.29	23.50	38.25	55.17	27.59	41.38
			VGen	31.18	26.76	32.35	53.42	34.36	53.82	63.17	36.81	61.72	70.59	41.18	64.71
	CodeT5p	136K	RTLLM	21.55	13.79	12.24	32.25	20.42	30.63	38.56	25.04	37.55	44.83	34.48	37.93
			VGen	34.12	22.35	31.76	55.47	32.78	51.64	65.51	35.01	63.21	76.47	35.29	76.47
		32K	RTLLM	1.21	0.34	0.00	3.87	1.54	0.00	5.15	2.63	0.00	6.90	3.45	0.00
			VGen	14.41	0.29	1.76	19.79	1.47	6.48	19.97	2.94	11.15	23.53	5.88	17.65
		64K	RTLLM	0.86	1.90	0.52	3.95	4.89	2.07	6.99	6.08	3.09	10.34	6.90	3.45
			VGen	15.88	10.59	7.06	21.09	13.23	13.54	22.91	14.71	16.22	23.53	17.65	17.65
		96K	RTLLM	5.00	0.69	0.69	11.91	2.93	3.09	15.46	5.17	5.26	17.24	10.34	6.90
			VGen	14.71	9.71	7.65	23.24	11.76	12.47	29.47	11.76	14.67	35.29	11.76	17.65
		128K	RTLLM	5.52	0.52	0.34	14.58	2.40	1.54	19.82	4.36	2.63	27.59	6.90	3.45
			VGen	15.29	9.71	9.41	21.57	11.76	11.74	23.34	11.76	11.76	23.53	11.76	11.76
Syntax	CodeLlama	34K	RTLLM	60.52	14.31	40.69	80.61	30.67	70.47	84.52	39.28	77.85	86.21	44.83	82.76
			VGen	86.76	69.71	88.82	99.14	80.61	99.48	99.97	82.10	99.97	100.00	88.24	100.00
		68K	RTLLM	60.69	26.90	53.45	79.48	51.12	75.87	84.87	62.85	81.10	89.66	68.97	82.76
			VGen	97.65	71.76	60.88	100.00	82.40	89.90	100.00	86.48	93.73	100.00	88.24	94.12
		102K	RTLLM	66.55	36.72	45.52	84.10	62.56	72.08	88.82	72.48	78.80	89.66	79.31	82.76
			VGen	96.47	66.18	75.59	100.00	78.82	97.14	100.00	81.64	99.80	100.00	88.24	100.00
	CodeT5p	136K	RTLLM	66.38	39.48	33.28	80.97	61.11	64.90	84.46	67.88	74.84	86.21	72.41	79.31
			VGen	99.12	67.65	73.53	100.00	79.89	96.94	100.00	82.65	99.78	100.00	88.24	100.00
		32K	RTLLM	12.59	9.66	10.86	33.97	22.50	32.60	46.19	28.52	45.12	58.62	34.48	51.72
			VGen	63.53	17.06	34.12	85.74	47.09	71.55	90.86	59.36	83.75	94.12	70.59	94.12
		64K	RTLLM	28.28	16.21	15.17	52.27	35.71	38.19	61.96	43.35	51.73	68.97	51.72	65.52
			VGen	74.12	46.76	26.47	86.85	77.14	64.50	88.14	84.70	79.40	88.24	88.24	88.24
		96K	RTLLM	33.28	12.76	24.83	59.49	29.05	46.58	67.72	38.94	59.44	72.41	48.28	68.97
			VGen	82.35	42.94	52.65	97.79	72.81	83.97	99.93	81.68	92.84	100.00	88.24	94.12
		128K	RTLLM	44.83	10.69	19.48	62.67	24.93	37.30	72.54	33.22	46.31	79.31	41.38	55.17
			VGen	77.35	43.82	35.88	98.00	75.25	71.91	99.89	84.38	82.89	100.00	94.12	88.24

TABLE II
EVALUATION RESULTS FOR THE SPEED OF GENERATING VERILOG CODE

Method	CodeLlama		CodeT5p	
	Speed (tokens/s)	Speedup	Speed (tokens/s)	Speedup
Ours	420.13	5.05	243.70	2.66
Medusa	294.99	3.55	106.33	1.16
NTP	83.13	1	91.65	1

further illustrate the effectiveness of our method, Fig. 5 compares the decoding process of our approach to those of Medusa and NTP for a specific example. Notably, our method generates the output in significantly fewer steps while maintaining the integrity of the syntactic structure at each decoding step.

The output quality of models fine-tuned with varying amounts of training data on the RTLLM [16] and VGen [22] benchmarks is summarized in Table I. The highest values for each benchmark, within the same model architecture and across different training data sizes and methods, are highlighted in bold. For clearer visualization, Fig. 6 highlights the pass@5 results of our method compared to the baselines using the CodeT5p architecture.

The models trained with our method show significant improvements in both functional and syntactic accuracy compared to those trained with Medusa. On the VGen benchmark, our method achieves a maximum functional accuracy increase of 30.5% using the pass@10 metric. For syntactic accuracy, it delivers a substantial improvement of 49.94% in the pass@5 metric on the RTLLM benchmark. When compared to the models fine-tuned with NTP, our approach achieves a functional accuracy improvement of up to 17.19% using the pass@10 metric on the RTLLM benchmark and a syntactic accuracy gain of up to 47.65% in the pass@1 metric on the VGen benchmark. Averaged across all benchmarks and evaluation metrics, models trained with

our method show a 13.03% improvement in functional accuracy over Medusa and a 5.91% improvement over NTP. For syntactic accuracy, our method demonstrates an average enhancement of 22.9% over Medusa and 11.8% over NTP. Additionally, our approach’s strong performance on small datasets highlights its effectiveness, achieving competitive results without requiring extensive additional data.

V. CONCLUSION

In this work, we introduce a novel application of speculative decoding for Verilog code generation, demonstrating its potential to enhance both inference speed and output quality. By aligning decoding stops with syntactically significant tokens extracted from ASTs, our method addresses the limitations of conventional tokenization and grammar-based approaches, enabling models to more effectively capture Verilog’s structural and semantic intricacies. The proposed method achieves significant advancements in generating both syntactically and functionally correct code while substantially accelerating inference, delivering up to a 17.19% improvement in pass@10 functional accuracy on the RTLLM benchmark and achieves up to a 5.05× speedup in Verilog code generation compared to the conventional NTP scheme. Furthermore, our approach enhances model inference speed by 1.42–2.29× over the original MEDUSA method. These results underscore the efficacy of leveraging syntax-enriched speculative decoding to unlock new possibilities for applying LLMs to specialized programming languages, paving the way for more efficient and reliable Verilog code generation.

VI. ACKNOWLEDGMENTS

This work was supported in part by the General Research Fund of the Hong Kong Research Grants Council (RGC) under Grant No. 14212422 and 14202824, and in part by National Technology Innovation Center for EDA.

REFERENCES

- [1] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 8696–8708.
- [2] Y. Wang, H. Le, A. Gotmare, N. Bui, J. Li, and S. Hoi, "Codet5+: Open code large language models for code understanding and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 1069–1088.
- [3] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
- [4] Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma *et al.*, "Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence," *arXiv preprint arXiv:2406.11931*, 2024.
- [5] R. Sennrich, "Neural machine translation of rare words with subword units," *arXiv preprint arXiv:1508.07909*, 2015.
- [6] M. Rabinovich, M. Stern, and D. Klein, "Abstract syntax networks for code generation and semantic parsing," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, pp. 1139–1149.
- [7] Z. Sun, Q. Zhu, L. Mou, Y. Xiong, G. Li, and L. Zhang, "A grammar-based structural cnn decoder for code generation," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 7055–7062.
- [8] Z. Sun, Q. Zhu, Y. Xiong, Y. Sun, L. Mou, and L. Zhang, "Treegen: A tree-based transformer architecture for code generation," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 8984–8991.
- [9] Y. Xiong and B. Wang, "L2s: A framework for synthesizing the most probable program under a specification," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 3, pp. 1–45, 2022.
- [10] Q. Zhu, Z. Sun, W. Zhang, Y. Xiong, and L. Zhang, "Grape: Grammar-preserving rule embedding," in *IJCAI*, 2022, pp. 4545–4551.
- [11] Q. Zhu, Q. Liang, Z. Sun, Y. Xiong, L. Zhang, and S. Cheng, "Grammart5: Grammar-integrated pretrained encoder-decoder neural model for code," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [12] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*. PMLR, 2023, pp. 19 274–19 286.
- [13] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.
- [14] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi *et al.*, "Specinfer: Accelerating large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 932–949.
- [15] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple llm inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024.
- [16] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "Rtllm: An open-source benchmark for design rtl generation with large language model," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 722–727.
- [17] H. Pearce, B. Tan, and R. Karri, "Dave: Deriving automatically verilog from english," in *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*, 2020, pp. 27–32.
- [18] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 2023, pp. 1–6.
- [19] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking large language models for automated verilog rtl code generation," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2023, pp. 1–6.
- [20] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Verilogval: Evaluating large language models for verilog code generation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–8.
- [21] K. Chang, Y. Wang, H. Ren, M. Wang, S. Liang, Y. Han, H. Li, and X. Li, "Chipgpt: How far are we from natural language hardware design," *arXiv preprint arXiv:2305.14019*, 2023.
- [22] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 3, pp. 1–31, 2024.
- [23] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, "Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution," in *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 2024, pp. 1–5.
- [24] K. Chang, K. Wang, N. Yang, Y. Wang, D. Jin, W. Zhu, Z. Chen, C. Li, H. Yan, Y. Zhou *et al.*, "Data is all you need: Finetuning llms for chip design via an automated design-data augmentation framework," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [25] Y. Zhang, Z. Yu, Y. Fu, C. Wan, and Y. C. Lin, "Mg-verilog: Multi-grained dataset towards enhanced llm-assisted verilog generation," in *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 2024, pp. 1–5.
- [26] F. Cui, C. Yin, K. Zhou, Y. Xiao, G. Sun, Q. Xu, Q. Guo, D. Song, D. Lin, X. Zhang *et al.*, "Origen: Enhancing rtl code generation with code-to-code augmentation and self-reflection," *arXiv preprint arXiv:2407.16237*, 2024.
- [27] Y. Zhao, D. Huang, C. Li, P. Jin, Z. Nan, T. Ma, L. Qi, Y. Pan, Z. Zhang, R. Zhang *et al.*, "Codev: Empowering llms for verilog generation through multi-level summarization," *arXiv preprint arXiv:2407.10424*, 2024.
- [28] X. Wang, Y. Wang, F. Mi, P. Zhou, Y. Wan, X. Liu, L. Li, H. Wu, J. Liu, and X. Jiang, "Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation," *arXiv preprint arXiv:2108.04556*, 2021.
- [29] X. Jiang, Z. Zheng, C. Lyu, L. Li, and L. Lyu, "Treebert: A tree-based pre-trained model for programming language," in *Uncertainty in Artificial Intelligence*. PMLR, 2021, pp. 54–63.
- [30] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unixcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 7212–7225.
- [31] Z. Yan, J. Liu, G. Li, Z. Han, and S. Qiu, "Privmin: Differentially private minhash for jaccard similarity computation," *arXiv preprint arXiv:1705.07258*, 2017.
- [32] X. Chen, Y. Meng, and G. Chen, "Incremental verilog parser," in *2023 International Symposium of Electronics Design Automation (ISED)*. IEEE, 2023, pp. 236–240.
- [33] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [34] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [35] Axolotl, "Axolotl," <https://github.com/OpenAccess-AI-Collective/axolotl>, 2023.
- [36] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *arXiv preprint arXiv:2305.14314*, 2023.
- [37] S. Williams, "Icarus verilog," <http://iverilog.icarus.com>, 2024, accessed: 2024-11-19.